

avito.tech 

ПОВЕДЕНЧЕСКИЕ ПАТТЕРНЫ

Афанасьев Юра

Отвечают за взаимодействия
объектов и классов
между собой

Решаемые задачи

- Распределяют обязанности и связи между объектами
- Реализуют алгоритмы поведения

1. **Chain of Responsibility.**
Цепочка обязанностей

2. **Command.** Команда

3. **Interpreter.** Интерпретатор

4. **Iterator.** Итератор

5. **Mediator.** Посредник

6. **Memento.** Хранитель

7. **Observer.** Наблюдатель

8. **State.** Состояние

9. **Strategy.** Стратегия

10. **Template Method.**
Шаблонный метод

11. **Visitor.** Посетитель

STRATEGY



СТРАТЕГИЯ

Определяет семейство алгоритмов,
инкапсулирует каждый из них
и делает их взаимозаменяемыми

Позволяет изменять алгоритмы
независимо от клиентов,
которые ими пользуются

Способы расширения класса

- Наследование (статическая связь)
- Композиция (динамическая связь)
- Декоратор (динамическая связь)
- Стратегия (динамическая связь)

Строковое
сравнение

Числовое
сравнение

Сравнение
без учёта регистра

```
sort($array, SORT_REGULAR);
```

Решаемая задача

- Позволяет использовать один из вариантов некоторого алгоритма
- Инкапсулирует сложные и/или специфичные части кода
- Задаёт классу определённое поведение для конкретного состояния системы
- Уменьшает количество логических операторов (if/switch) благодаря переносу поведения в специальные классы

```
// ...код...
```

```
if ($isClientBirthday) {  
    $discount += 10;  
} elseif ($isVipClient) {  
    $discount += 5;  
} else {  
    $discount += 0;  
}
```

```
// ...код...
```



```
// ...код...
```

```
$discount += $specialOffer->get();
```

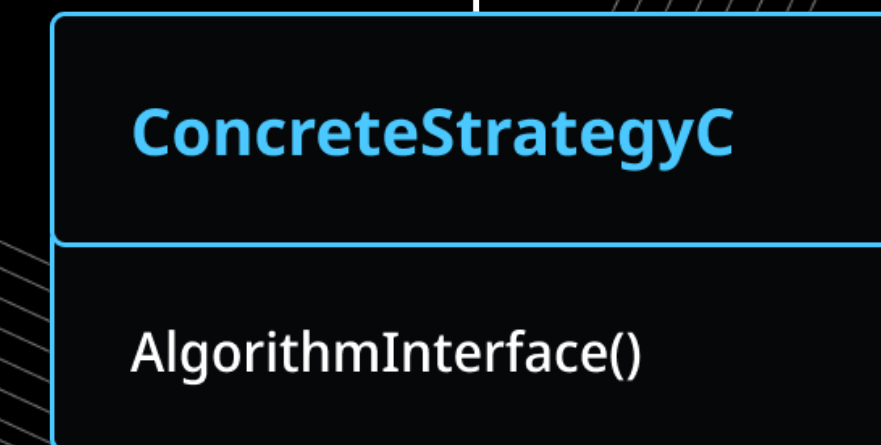
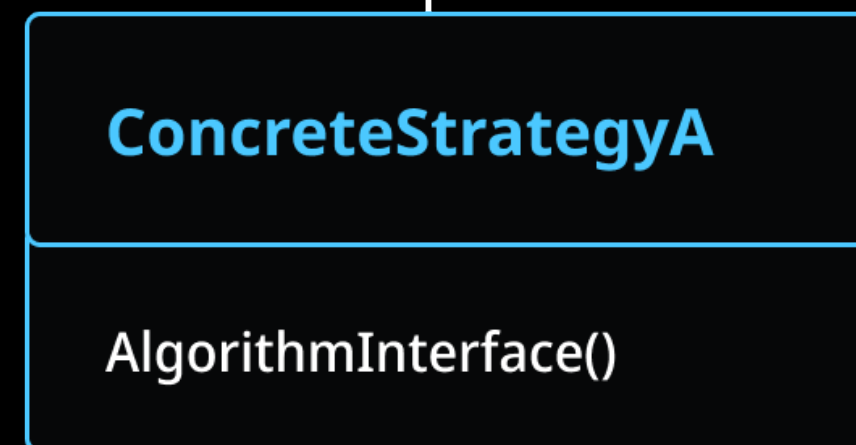
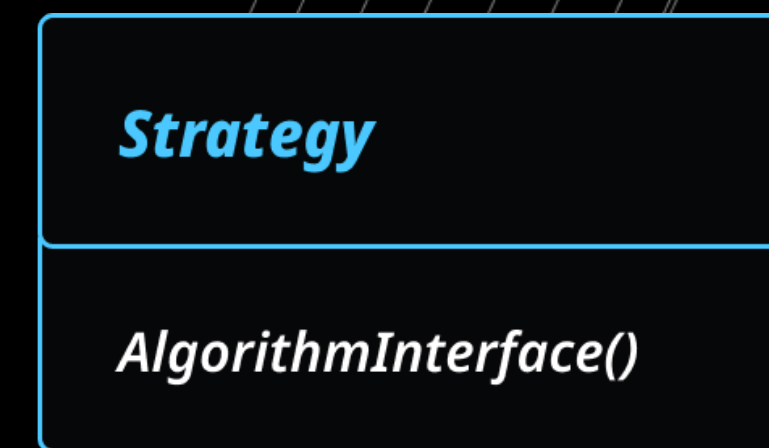
```
// ...код...
```

```
new BirthdayOffer()  
new VipOffer()  
new NullOffer()
```


Контекст
(работает с конкретным
алгоритмом)



Интерфейс
стратегии



Реализации
стратегий

Контекст



Расширяется стратегией



```
class Context
{
    public function someMethod(Strategy $strategy)
    {
        echo 'Контекст. ' . $strategy->algorithm();
    }
}
```

Интерфейс стратегии
(унифицирует сигнатуру)

```
interface Strategy
{
    public function algorithm();

    // public function anotherAlgorithm();
}
```

Методов может
быть много

```
class ConcreteStrategyA implements Strategy
```

```
{  
    public function algorithm()  
    {  
        return 'Выполнение алгоритма А.' . PHP_EOL;  
    }  
}
```

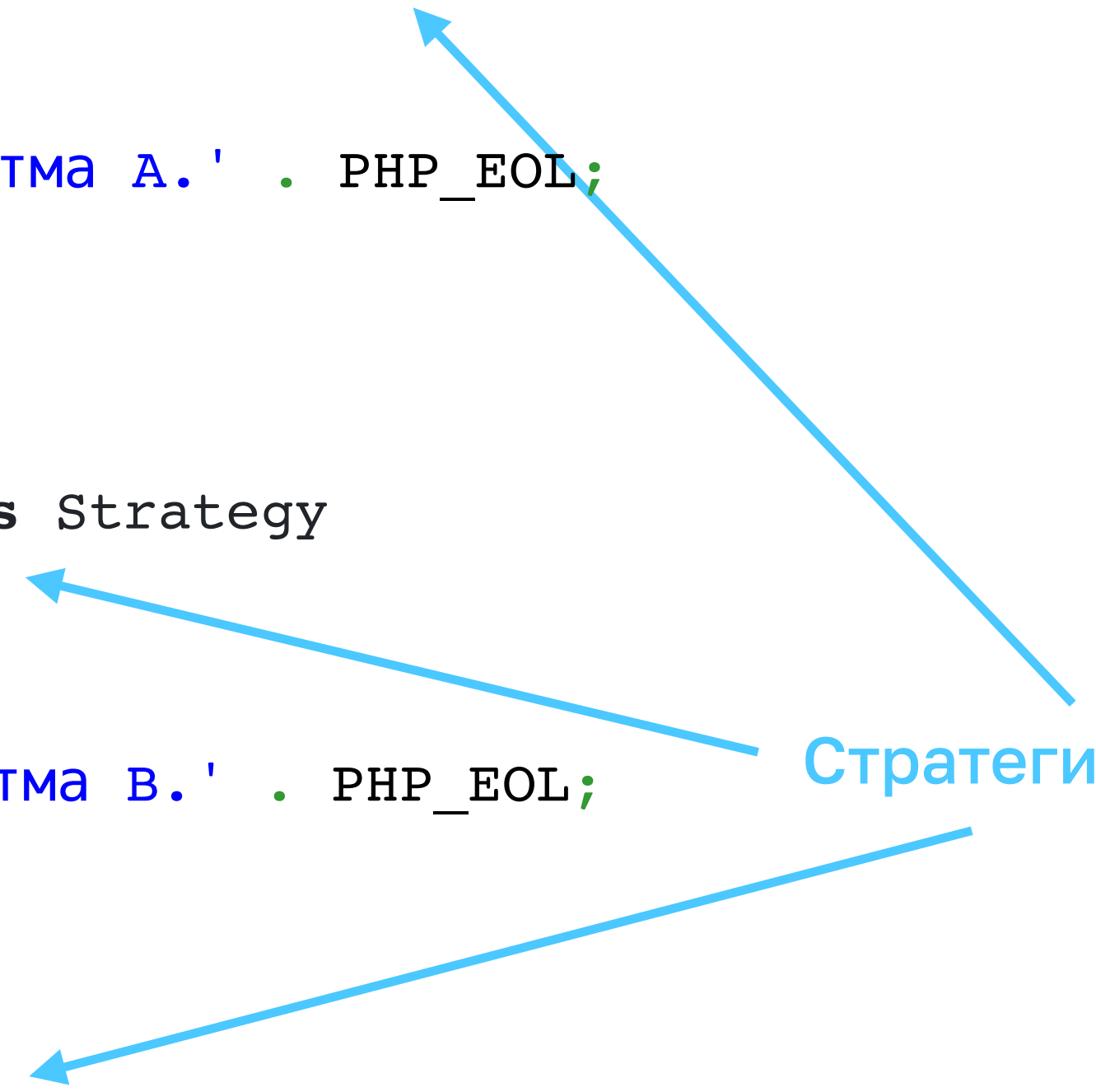
```
class ConcreteStrategyB implements Strategy
```

```
{  
    public function algorithm()  
    {  
        return 'Выполнение алгоритма В.' . PHP_EOL;  
    }  
}
```

```
class ConcreteStrategyC implements Strategy
```

```
{  
    public function algorithm()  
    {  
        return 'Выполнение алгоритма С.' . PHP_EOL;  
    }  
}
```

Стратегии




```
testStrategy(new ContextStrategyA());  
testStrategy(new ContextStrategyB());  
testStrategy(new ContextStrategyC());
```

В зависимости
от стратегии
получаем разные
результаты

```
function testStrategy(strategy $strategy)  
{  
    $context = new Context();  
    $context->someMethod($strategy);  
}
```

Плюсы

- Поведение не зашивается жёстко, поэтому его легко заменить или расширить
- Помогает избавиться от условных операторов в сильно ветвистой логике
- Альтернатива наследованию
- Возможно выбрать оптимальный алгоритм для одного и того же поведения

Минусы



- Необходимо разобраться в отличиях стратегий, чтобы подобрать оптимальный вариант
- Применение паттерна увеличивает число классов
- Имея единый интерфейс, методы стратегии могут получать неиспользуемые параметры

Параметры
для вычисления скидки

```
class VipStrategy implements IDiscount
{
    public function getDiscount(IUser $user, IProducts $products)
    {
        // ВЫЧИСЛЕНИЕ СКИДКИ НА ОСНОВЕ $user И $products
        return $discount;
    }
}
```

Бесполезные
параметры для метода

```
class NullStrategy implements IDiscount
{
    public function getDiscount(IUser $user, IProducts $products)
    {
        return 0;
    }
}
```

Почему Стратегия - поведенческий паттерн?

- Выделяет обязанности (задачи) в другие классы
- Реализует алгоритмы поведения

**Инкапсулирует
алгоритмы
и/или поведение
в специальные
классы**

Strategy

- Имеет множество сущностей
- Выбор стратегии происходит в runtime

DI

- Реализация внедряемого класса чаще всего одна
- Подменить внедряемый класс в runtime невозможно

Decorator

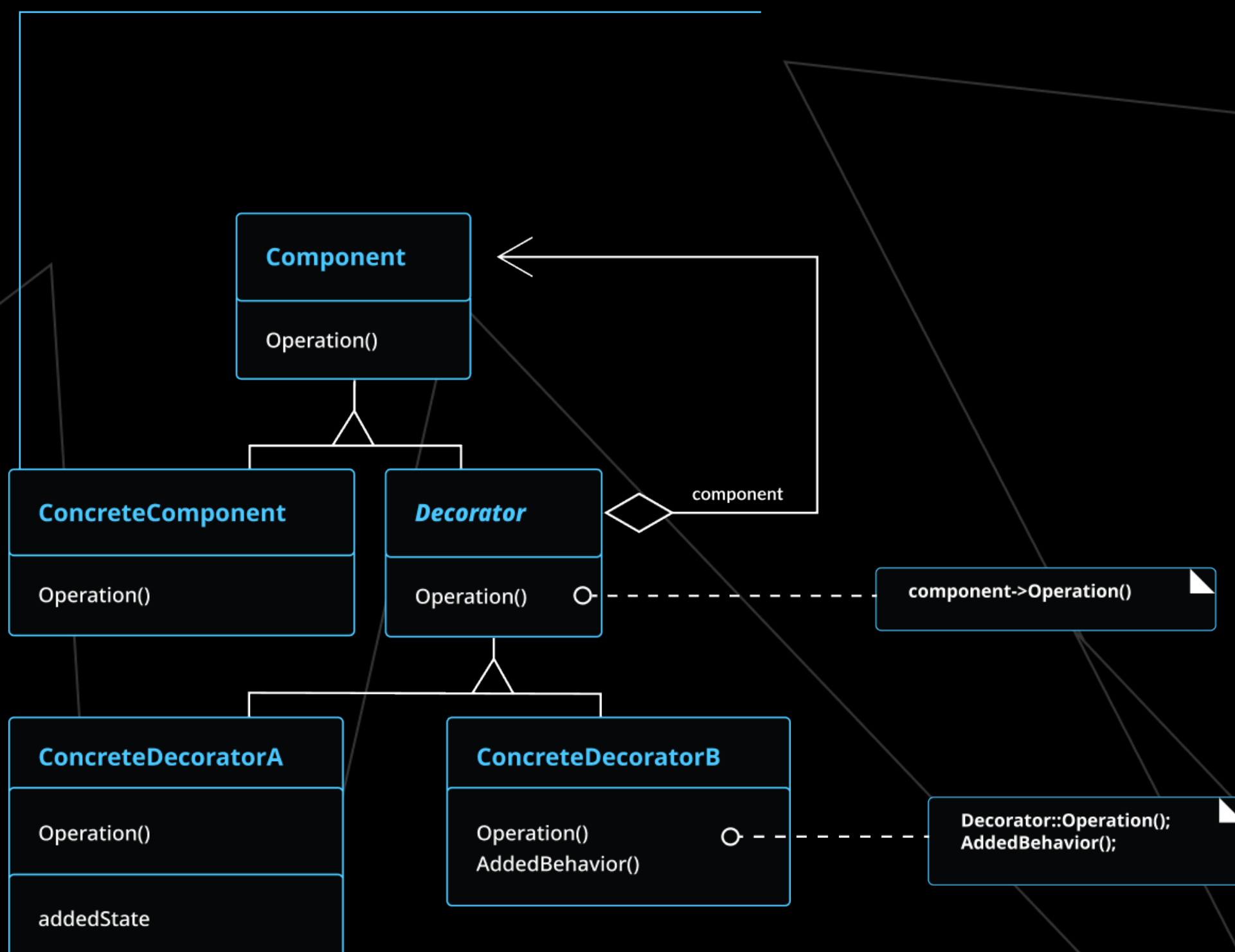
- Позволяет изменить внешний облик (поведение) объекта

Strategy

- Позволяет изменить внутреннее содержание (поведение) объекта

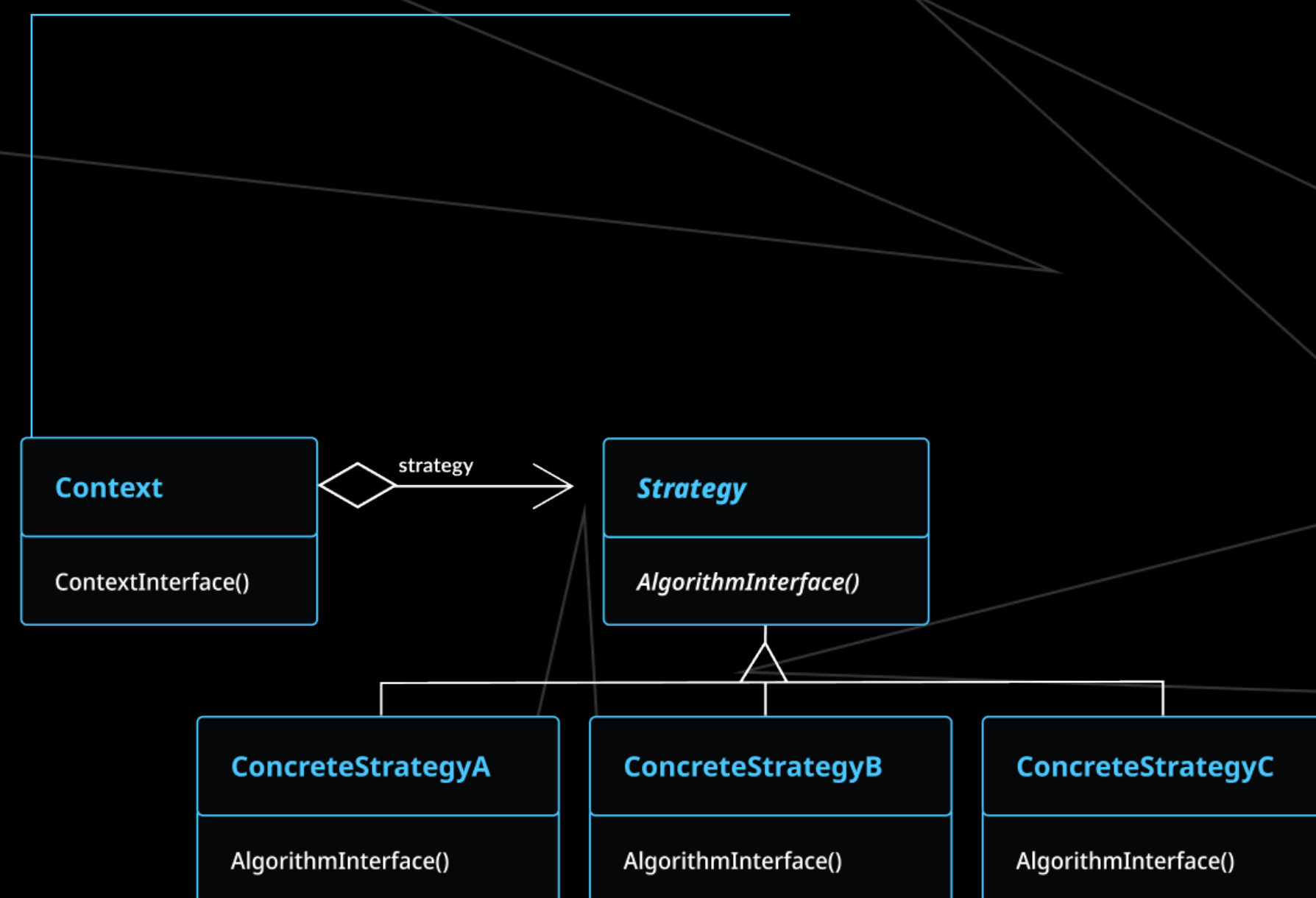
Decorator

Неизвестно, какие
классы добавят обязанности



Strategy

Основана
на композиции объектов



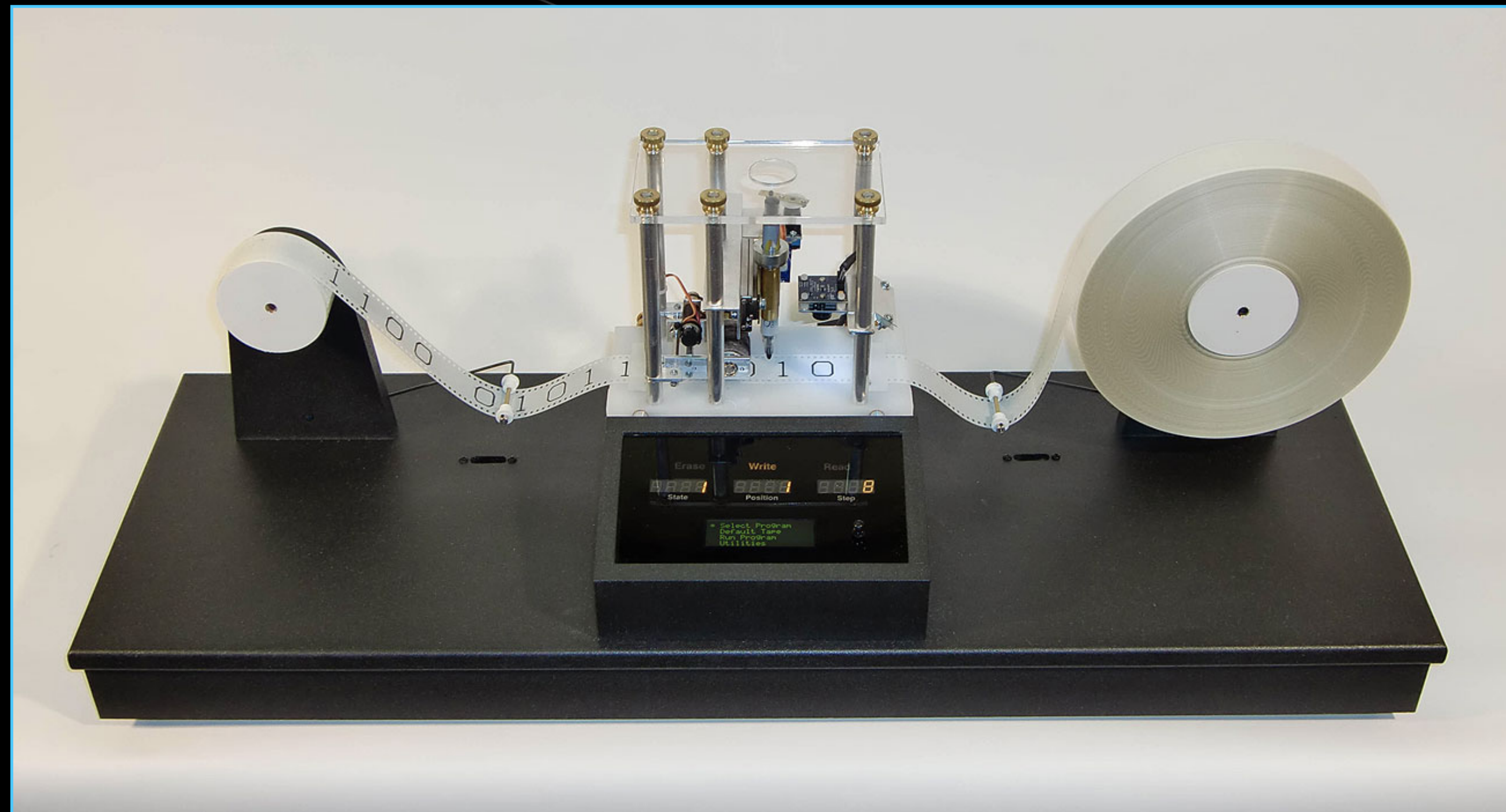
STATE



СОСТОЯНИЕ

Позволяет объекту варьировать
свое поведение в зависимости
от внутреннего состояния

Извне создается впечатление,
что изменился класс объекта



Машина Тьюринга

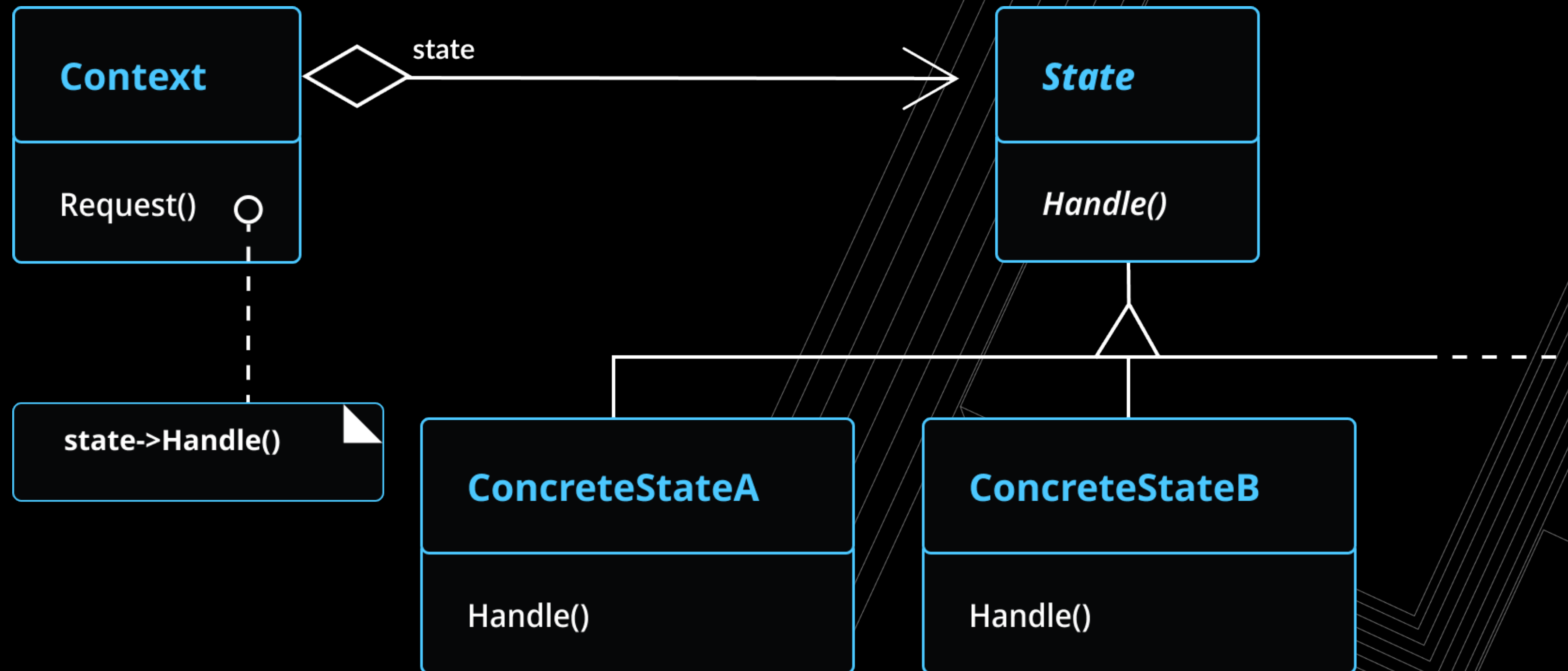
Одна из реализаций конечного автомата

Решаемая задача

- Изменяет поведение объекта в зависимости от его состояния
- Убирает из кода условные операторы, оперирующие состоянием объекта

Контекст
(определяет базовое поведение)

Состояние



Конкретные состояния
(реализуют поведения)

Контекст

Хранит ссылку
на текущее состояние

```
class Context
```

```
{
```

```
    private State $state;
```

```
    // private OtherState $otherState;
```

```
    public function __construct()
```

```
    {
```

```
        $this->state = new ConcreteStateA();
```

```
    }
```

```
    public function request()
```

```
    {
```

```
        // ...КОД...
```

```
        $this->state->handle($this);
```

```
    }
```

```
    public function setState(State $state)
```

```
    {
```

```
        $this->state = $state;
```

```
    }
```

```
}
```

Задаём первоначальное
состояние контекста

Основной метод
взаимодействия

Передаём состояниям
ссылку на себя

Позволяет изменять
состояние контекста

Интерфейс определяющий
контракт состояниям

Реализует
конкретное состояние

```
interface State
{
    public function handle(Context $context);

    // public function otherHandle(Context $context);
}
```


Конкретные
состояния

```
class ConcreteStateA implements State
{
    public function handle(Context $context)
    {
        echo 'Состояние А.' . PHP_EOL;
        $context->setState(new ConcreteStateB);
    }
}
```

Имея ссылку на контекст,
можно переопределить
состояние

```
class ConcreteStateB implements State
{
    public function handle(Context $context)
    {
        echo 'Состояние В.' . PHP_EOL;
        $context->setState(new ConcreteStateC);
    }
}
```

```
class ConcreteStateC implements State
{
    public function handle(Context $context)
    {
        echo 'Состояние С.' . PHP_EOL;
        $context->setState(new ConcreteStateA);
    }
}
```


Создаём контекст

```
function testState()  
{  
    $context = new Context();  
  
    $context->request();  
  
    $context->request();  
  
    $context->request();  
}
```

Выводит: Состояние А.
И меняет на состояние В

Выводит: Состояние В.
И меняет на состояние С

Выводит: Состояние С.
И меняет на состояние А

Особенности

- Клиенты оперируют контекстом
- Только контекст или конкретные состояния решают как происходит смена состояний
- Конкретные состояния могут получить доступ к контексту

Плюсы

- Все состояния выделены в отдельные классы
- Контексты не являются чем-то монолитным
- Просто добавить новое состояние и поведение
- Нет однотипных условных операторов в контексте

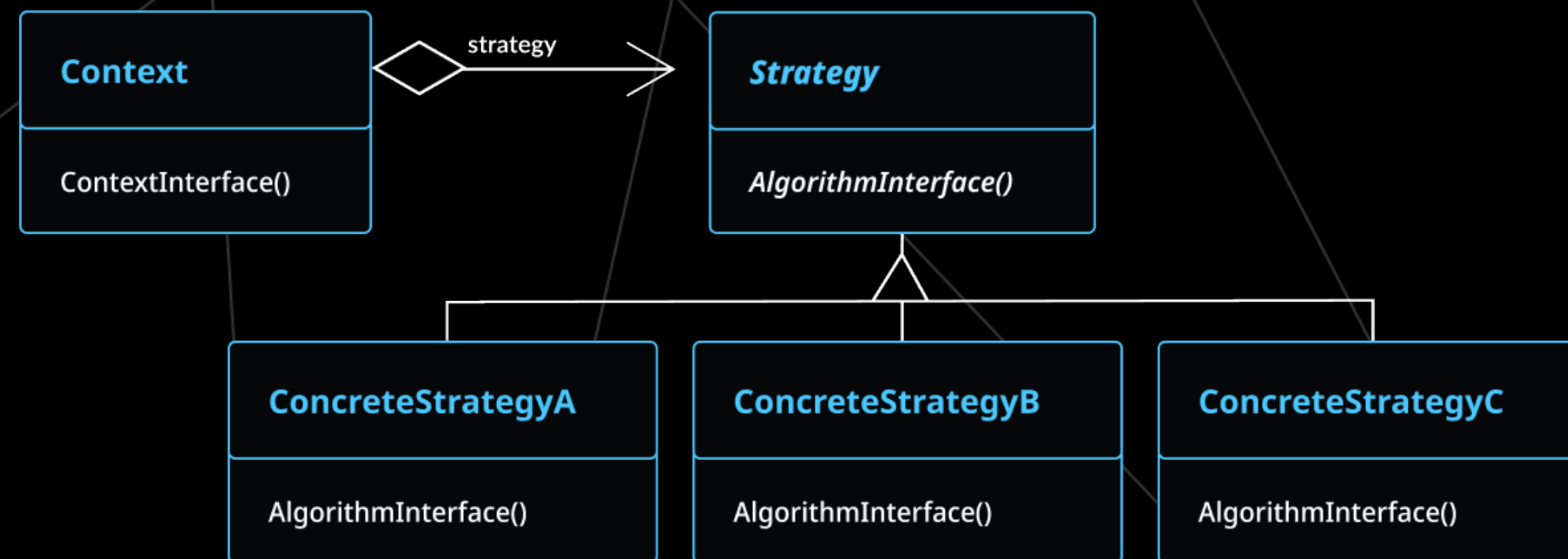
Минусы



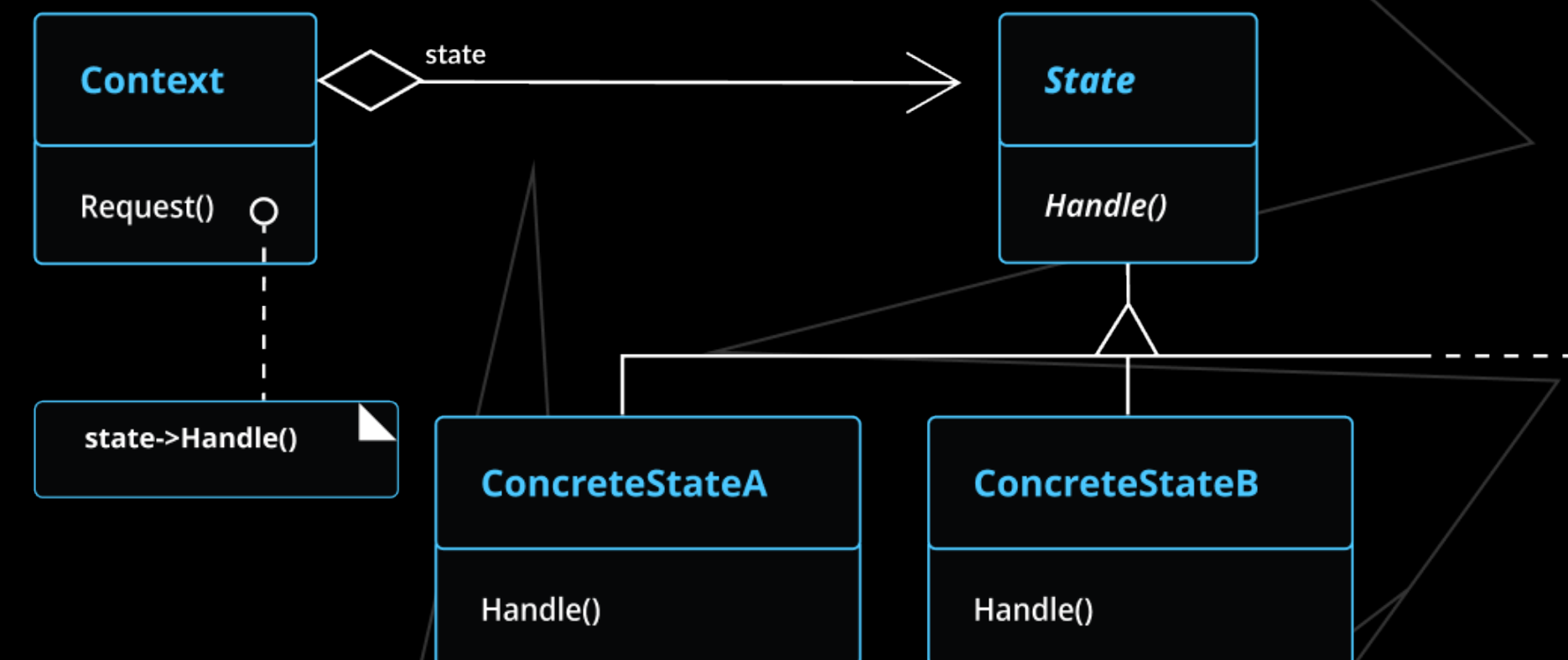
- Большое число классов
- Если состояний мало, то реализация будет громоздкой

Инкапсулирует проверки
состояния объекта
для изменения его поведения

Strategy



State



State

Strategy

- Использует композицию для изменения поведения
- Стратегию выбирает клиент
- Объекты-помощники не знают о Контексте и лишь выполняют дополнительное поведение

- Использует композицию для изменения поведения
- Состояние меняется внутри контекста
- Объекты-помощники занимаются изменением состояния Контекста

avito.tech

Смотрите все серии на сайте
<https://avito.tech/patterns>

